Iteration Zero – k-NN Model

1.Introduction

This notebook presents the first machine-learning prototype for the PlaySafe project.

The aim of Iteration Zero is to build a simple k-Nearest Neighbors (k-NN) model that predicts whether a LaLiga player will suffer at least one injury in the following season.

The purpose of this iteration is to create a first working prototype and obtain initial predictions.

The cleaned dataset produced during the data-cleaning stage is used as the starting point for the modelling process.

The cleaned dataset created during the data-cleaning stage is loaded into the notebook. This dataset contains player information, season information, playing time, and injury records.

In [119…

```
import pandas as pd
import numpy as np

df = pd.read_csv("../data/processed/laliga_cleaned.csv")

print("Shape:", df.shape)
```

```
Shape: (3713, 14)
```

2.Prepare data

The original dataset can contain multiple injury records for the same player and season. For example, a player may have several injuries during one season.

For the machine-learning model, one row per player-season is required. Therefore, the injury records are aggregated so that has_injury indicates whether the player had at least one recorded injury during that season.

In [ ]:

```
# Create one row per player and season

player_season = (
    df.groupby(["player_name", "season"], as_index=False)
      .agg({
          "height": "first",
          "position": "first",
          "current_age_x": "first",
          "minutes_played": "first",
          "injury": lambda x: x.notna().any() #true if the player has at least o
      })
)
```

```
player_season = player_season.rename(columns={
    "injury": "has_injury"
})

print("Shape:", player_season.shape)
player_season.head(10)
```

```
Shape: (2942, 7)
```

Out[ ]:

	player_name	season	height	position	current_age_x	minutes_played	has_injury
0	Abde Ezzalzouli	19/20	177.0	Left Winger	23	160.0	False
1	Abde Ezzalzouli	20/21	177.0	Left Winger	23	1788.0	False
2	Abde Ezzalzouli	21/22	177.0	Left Winger	23	3769.0	True
3	Abde Ezzalzouli	22/23	177.0	Left Winger	23	2583.0	True
4	Abde Ezzalzouli	23/24	177.0	Left Winger	23	3016.0	False
5	Abde Ezzalzouli	24/25	177.0	Left Winger	23	3253.0	False
6	Abdel Abqar	18/19	188.0	Centre-Back	25	3394.0	False
7	Abdel Abqar	20/21	188.0	Centre-Back	25	2388.0	False
8	Abdel Abqar	21/22	188.0	Centre-Back	25	45.0	False
9	Abdel Abqar	22/23	188.0	Centre-Back	25	3700.0	False

3. Create the Target Variable

The target variable is `has_injury_next_season`.

For each player-season, the target indicates whether the same player has at least one recorded injury in the following season.

The following season is identified explicitly from the season value. Player-seasons for which the following season is not available are removed because a target value cannot be determined for them.

In [ ]:

```
# Create a separate table containing each player current season
# and whether they had an injury in the next season
next_season = player_season[[
    "player_name", "season", "has_injury"
]].copy()
```

```
#calculate the following season
next_season["season_start"] = (
    next_season["season"].str[:2].astype(int)
)

next_season["next_season"] = (
    (next_season["season_start"] + 1).astype(str).str.zfill(2)
    + "/"
    + (next_season["season_start"] + 2).astype(str).str.zfill(2)
)

next_season = next_season.rename(columns={
    "has_injury": "has_injury_next_season"
})

# match each player per season with the same player in the next season and wheth
player_season = player_season.merge(
    next_season[[
        "player_name",
        "next_season",
        "has_injury_next_season"
    ]],
    left_on=["player_name", "season"],
    right_on=["player_name", "next_season"],
    how="left"
)
#remove the column if next season isnt available
player_season = player_season.dropna(
    subset=["has_injury_next_season"]
)
#convert the target to an int (0 or 1)
player_season["has_injury_next_season"] = (
    player_season["has_injury_next_season"].astype(int)
)

print("Shape:", player_season.shape)
```

```
Shape: (2082, 9)
```

In [ ]:
```
print(player_season["has_injury_next_season"].value_counts())
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 1
----> 1 print(player_season["has_injury_next_season"].value_counts())

NameError: name 'player_season' is not defined
```

4. Prepare the Features

For the first prototype, a small number of basic player and playing-time variables are used:

- height
- position
- current age
- minutes played

The player name is not used as a predictive feature because it is an identifier.

Injury information from the target season is also not used as a predictor because this information would not be available when making a prediction for the following season. The target variable is separated from the input features as X and y.

In [123…
```
features = [
    "height",
    "position",
    "current_age_x",
    "minutes_played"
]

X = player_season[features]
y = player_season["has_injury_next_season"]
```

5. Train-Test Split

Because the objective is to predict injuries in a future season, the data is split chronologically rather than randomly.

The model is trained using earlier seasons from 13/14 to 21/22. The later seasons from 22/23 to 24/25 are kept as unseen test data.

In [124…
```
# Sort seasons chronologically
player_season = player_season.sort_values("season")

# Check the available seasons
print(player_season["season"].unique())
```

```
<StringArray>
['13/14', '14/15', '15/16', '16/17', '17/18', '18/19', '19/20', '20/21',
 '21/22', '22/23', '23/24', '24/25']
Length: 12, dtype: str
```

In [ ]:
```
train = player_season[player_season["season"] <= "21/22"].copy()
test = player_season[player_season["season"] > "21/22"].copy()
# use aerlier season to train and later seasons to test
print("Training set:", train.shape)
print("Test set:", test.shape)
```

```
Training set: (1411, 9)
Test set: (671, 9)
```

In [126…
```
print("Training seasons:", train["season"].unique())
print("Test seasons:", test["season"].unique())
```

```
Training seasons: <StringArray>
['13/14', '14/15', '15/16', '16/17', '17/18', '18/19', '19/20', '20/21',
 '21/22']
Length: 9, dtype: str
Test seasons: <StringArray>
['22/23', '23/24', '24/25']
Length: 3, dtype: str
```

6. Preprocess the Features

Before training the k-NN model, the numerical features are scaled and the categorical `position` variable is converted into numerical values.

This preprocessing is necessary because k-NN is based on distances between observations.

In [ ]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

X_train = train[features]
y_train = train["has_injury_next_season"]

X_test = test[features]
y_test = test["has_injury_next_season"]

numeric_features = [
    "height",
    "current_age_x",
    "minutes_played"
]

categorical_features = [
    "position" # this one need to be encoded
]

numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")), #fill missing values
        ("scaler", StandardScaler()) #scales the values so they dont dominate th
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")), #fills in with the
        ("encoder", OneHotEncoder(handle_unknown="ignore")) #encodes the positio
    ]
)

preprocessor = ColumnTransformer( # preprocess each type of feature
    transformers=[
        ("num", numeric_pipeline, numeric_features),
        ("cat", categorical_pipeline, categorical_features)
    ]
)
```

7. Train the k-NN Model

A k-Nearest Neighbors (k-NN) classifier is used as the first machine-learning model.

The model is kept at a basic setting without hyperparameter tuning, as required for Iteration Zero. I will use a simple value of k = 5

In [ ]:
```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5) #model considers the 5 nearest neighbo

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("knn", knn)
    ]
)

model.fit(X_train, y_train)
print("Model trained successfully.")
```

```
Model trained successfully.
```

8. Model Evaluation

After training, the model is used to make predictions on the unseen test data. The first prototype is evaluated using accuracy, precision, recall, and F1-score.

These metrics provide an initial indication of how well the model predicts injuries in the following season.

In [129…
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-score:", f1)
```

```
Accuracy: 0.5290611028315947
Precision: 0.6091954022988506
Recall: 0.1582089552238806
F1-score: 0.25118483412322273
```

Evaluation Results

The first prototype achieved an accuracy of **52.9%**, precision of **60.9%**, recall of **15.8%**, and an F1-score of **25.1%**.

The results indicate that the first prototype has limited predictive performance. In particular, the low recall shows that the model identifies only a relatively small proportion of the players who actually experience an injury in the following season.

However, this result is acceptable for Iteration Zero because the purpose of this stage is to establish a simple working baseline rather than to optimise the model. The results provide a starting point for identifying improvements in later iterations.

In [141…

```
# Example player
player_input = pd.DataFrame({
    "height": [70],
    "position": ["Middfielder"],
    "current_age_x": [62],
    "minutes_played": [2980]
})

# Make prediction
prediction = model.predict(player_input)[0]
probability = model.predict_proba(player_input)[0][1]

print("Prediction:", prediction)
print("Probability of injury:", probability)
```

```
Prediction: 0
Probability of injury: 0.4
```

9. Conclusion

The first Iteration Zero prototype implemented a k-Nearest Neighbors model for predicting whether a LaLiga player will suffer at least one injury in the following season.

The modelling process included aggregating the data to player-season level, creating the future injury target, selecting a small set of features, splitting the data chronologically, preprocessing the variables, and training a k-NN classifier.

The model achieved an accuracy of 52.9%, precision of 60.9%, recall of 15.8%, and an F1-score of 25.1% on the test data. The low recall indicates that the first prototype is not yet effective at identifying players who will experience an injury.

Nevertheless, the objective of Iteration Zero was to create a simple working baseline rather than an optimised model. The results therefore provide a useful starting point for the next iteration, where the features, data preparation, and model configuration can be investigated and improved.